Project Directive 1: The Demogorgon Radar (Real-Time Multiplayer Hunt)

Story

Strange disturbances have been detected around the Hawkins research facility. Scientists believe a Demogorgon has escaped into the building. Using experimental radar devices on their phones, Hawkins Lab Security must track and avoid the creature moving through the venue.

Mission Brief

Build a real-time multiplayer location-based game where players move physically around the venue while their positions are tracked on a radar interface. One player is secretly assigned as the Demogorgon, while the others act as Hawkins Lab Security. If the Demogorgon gets close to a player, the system triggers alerts on their device.

Core Objectives

- Track players' real-time locations.
- Randomly assign one player as the Demogorgon.
- Detect proximity between players.
- Trigger alerts when the Demogorgon approaches.

[OPEN ENDED] Possible Features.

- **Multiplayer Lobby:** Players join the game from their phones before the match begins.
- **Secret Role Assignment:** One player becomes the Demogorgon while others play as Security Agents.
- **Radar Interface:** Players can see nearby activity as they move around the venue.
- **Proximity Alerts:** Phones vibrate, flash, or play warning sounds when the Demogorgon gets close.
- **Game End Conditions:** The game may end when the Demogorgon catches all players or when Security successfully identifies and captures it.

Project Directive 2: Hawkins Secure File Exchange

Story

In Hawkins, strange events have disrupted normal communication networks. Scientists at Hawkins National Laboratory must exchange sensitive research files about the Upside Down quickly and securely. However, internet connections are unreliable, and using cloud services could expose confidential information. To keep research data safe, Hawkins engineers need a local file transfer system that allows devices on the same network to share files directly without relying on the internet.

Mission Brief

Build an offline peer-to-peer file sharing web application that allows devices on the same local network to discover each other and transfer files securely without using cloud services. The system should work as a web app that enables quick and simple file transfers between nearby devices connected to the same network.

Core Objectives

- Enable devices on the same network to discover each other automatically.
- Allow users to send and receive files directly between devices.
- Ensure file transfers are secure and reliable.

[OPEN ENDED] Possible Features

- **Device Discovery:** Automatically detect nearby devices connected to the same local network.
- **Simple Connection Setup:** Allow users to connect to another device easily, possibly using a QR code or connection code.
- **Secure File Transfer:** Transfer files between devices using basic encryption or secure communication methods.
- **Simple Interface:** Provide a clean web interface where users can easily select and send files to nearby devices.

Project Directive 3: Mr. Clarke's Automated Briefing Generator

Story

Mr. Clarke, Hawkins' beloved science teacher, has mysteriously disappeared, leaving behind a collection of dense physics books, radio manuals, and research notes. The AV Club needs answers quickly to understand the strange signals appearing around Hawkins, but reading through hundreds of pages manually would take too long. A simple text terminal isn't enough; the party needs to visually brief the town sheriff and other allies. To solve this, the team wants a tool that can read all of Mr. Clarke's documents and automatically generate animated presentation decks summarizing the complex physics of the Upside Down.

Mission Brief

Build an AI-powered engine that ingests a folder of text or PDF documents and programmatically generates a fully formatted, animated presentation deck (such as a .pptx file or an HTML5 slide presentation like Reveal.js). A simple RAG (Retrieval-Augmented Generation) chat interface is **not enough**. The system must intelligently chunk the retrieved data, outline a logical presentation structure, and automatically output slides complete with bullet points and visual animations.

Core Objectives

- Ingest and process a local folder of reference documents.
- **AUTOMATED PRESENTATION OUTPUT:** Programmatically generate a multi-slide presentation (e.g., PowerPoint, Google Slides API, or HTML-based slides) based on user prompts or questions.
- **PROGRAMMATIC ANIMATIONS:** The generated presentation must include automated slide transitions and/or element animations (e.g., bullet points fading in sequentially).
- Retrieve relevant information from the documents to accurately populate the slides.

[OPEN ENDED] Possible Features

- **Q&A to Slide Deck:** Allow users to ask a question through a CLI or web interface, and have the system output a 5-slide animated deck answering it in detail.
- **Visual Asset Generation:** Use AI to generate relevant charts, graphs, or image placeholders on the generated slides.

- **Source Citation Slides:** Automatically generate a final "References" slide citing the specific pages of Mr. Clarke's documents used to create the presentation.

Project Directive 4: Cerebro's Code Red Synchronizer

Story

In Hawkins, the party has intercepted an urgent Code Red message related to activity from the Upside Down. Dustin must broadcast the warning video to everyone in the group hiding across the town. However, the message must be perfectly synchronized so that everyone sees the exact same frame at the exact same time, ensuring the warning reaches the team simultaneously.

Mission Brief

Build a web-based synchronized media player where one user acts as the Broadcaster and others join as Listeners. The Broadcaster controls playback while all connected users receive the same playback state in real time.

Core Objectives

- **RETRO UI THEME:** The application interface MUST follow a retro Stranger Things style, such as an 80s ham radio, Cerebro machine console, or Hawkins Lab control panel.
- Allow one user to act as the Broadcaster controlling playback.
- Allow multiple users to join as Listeners.
- Synchronize play, pause, and timeline actions across clients.
- Provide tools to verify synchronization and connection health.
- Use networking techniques such as STUN servers for peer discovery and NAT traversal if implementing peer-to-peer communication.

[OPEN ENDED] Possible Features

- **Broadcaster Control Panel:** Allow one user to control playback actions such as play, pause, and timeline scrubbing.
- **Real-Time Synchronization:** Use real-time communication (such as WebSockets or WebRTC) to synchronize playback across clients.
- **STUN Server Integration:** Use a STUN server when implementing WebRTC connections to help clients discover their public network addresses and establish peer-to-peer connections.

- **Visual Sync Indicator:** Display the video timecode on the player so synchronization across devices can be visually verified.
- **Network Debug Mode:** Provide an optional view showing connection status, network latency, or peer connection details.

Project Directive 5: The Hellfire Party's Ledger Resolver

Story

As the Party struggles with messy spreadsheets tracking their pooled money for Demodog supplies, Eddie Munson urgently needs an un-hackable ledger to prevent players from secretly altering their Hellfire Club D&D inventories. With Mayor Kline and the school cracking down on club activities, standard digital records are too risky. To solve both problems, they must build a hidden web portal that cleans and optimizes their scattered financial records, securely settling real-world debts and locking down virtual stats on an immutable blockchain network.

Mission Brief

Build a full-stack web application that processes a messy dataset of debts and calculates the most efficient way for everyone to settle their balances. The system should accept a CSV file, clean the data, and apply a graph-based optimization algorithm to minimize the number of transactions required.

Once calculated, the web interface must connect to a deployed Smart Contract managing a custom token supply (e.g., "Hellfire Gold"). Users should be able to connect their browser wallets and execute the settled transactions securely on a testnet directly from the web application.

Core Objectives

- **RETRO UI THEME:** The web application interface MUST follow an 80s retro theme, such as an arcade machine dashboard, a ham radio display, or a secret Hawkins Lab control panel, to keep the ledger "hidden" from authorities.
- Accept a CSV file containing debt or IOU records via the web interface.
- Clean and normalize messy or inconsistent data.
- Calculate the minimal set of transactions required to settle debts.
- Deploy an ERC-20 style token contract to a local or public testnet.
- Integrate a Web3 provider (like MetaMask) into the web application frontend.
- Execute token transfers between players via the web portal and display the transaction hash.

[OPEN ENDED] Possible Features

- **Data Cleaning Engine:** Automatically handle issues such as typos, missing values, duplicate entries, or inconsistent formatting upon CSV upload.
- **Debt Optimization Algorithm:** Use a graph-based or cash-flow minimization algorithm to compute the smallest number of transactions required to settle all balances.
- **Smart Contract & Web3 Integration:** Write and deploy a secure smart contract that handles minting and transferring custom tokens, allowing users to connect their wallets and sign transactions directly in the browser.
- **Transaction Resolution Output:** Display clear, step-by-step instructions showing who should pay whom and how much, along with real-time blockchain verification links (e.g., an Etherscan testnet link).